Academic Project Documentation

# Enforcing Mandatory Fields UsingUI Policies and Migrating ChangesWith Update Sets in ServiceNow

## Submitted By


| | |
|---|---|
| 04EAFB35E0E283CA14448B15F5326A30 | ABINESHA V |
| EF3805FD6D72A9D91274F6ED219A92B4 | YOKESH S |
| 9E7F8DEA853B1EB84E24FF1D888020EE | ABINAYA M |
| A022DDC40A7B69682B645F75200C077E | ATCHASIYA E |
| E28BC73DEAB38A2B83F442297AF52751 | BHARATHI S |


*Submitted in partial fulfilment of the requirements of the course*

*ServiceNow Platform Administration and IT Service Management*

# 1. Abstract

This project documents the design, implementation, and migration of a UI Policy withintheServiceNow platform, specifically addressing the enforcement of mandatory field completionon the Incident form. The central requirement is that the Priority field must become mandatorywhenever an incident's State is changed to "In Progress", thereby preventing agents fromadvancing an incident without supplying the information necessary for effective prioritisationand SLA management.

The implementation leverages ServiceNow's native UI Policy and UI PolicyActionframework, which operates entirely on the client side without customscripting. Allconfiguration changes are systematically captured in a dedicated Update Set, a version-controlled container that records every platform customisation made during a developmentsession. Once verified, the Update Set is exported as an XML file and imported into a separateServiceNow instance, where it is previewed for conflicts and committed to apply the identicalconfiguration.

The project demonstrates the complete lifecycle of a low-code ServiceNowconfiguration:from problem identification and requirement analysis through hands-on implementation, structured migration, and functional validation. The outcome confirms that mandatoryfieldenforcement can be achieved declaratively, that configuration changes can be reliablytransported between environments, and that data quality in incident management canbemeaningfully improved without writing a single line of server-side code.

The configuration changes are systematically captured within a dedicated Update Set, whichservesasa transportable container for all customisations made to the ServiceNow instance. The UpdateSet issubsequently exported as an XML file and imported into a separate target instance, demonstratingacomplete, version-controlled migration workflow. The project validates the effectiveness of bothmechanisms through structured testing within the Incident module, confirming correct enforcementbehaviour and successful cross-instance configuration transfer.

# 2. Introduction

A fundamental aspect of maintaining data quality within ServiceNow is the enforcement of businessrules at the form level. While server-side mechanisms such as Business Rules and Access Controlsarecommonly employed, they operate on the server after data submission. In contrast, UI Policiesareclient-side configurations that dynamically modify form behaviour in real time, based ondefinedconditions. This distinction makes UI Policies particularly effective for enforcing field-level requirements that must be communicated to the user before form

submission occurs.

Update Sets are a native ServiceNow mechanism that provides a structured approach to capturing, managing, and transporting configuration changes between instances. In an enterprise ServiceNowdeployment, configurations are typically developed in a sub-production development instance, validated in a test or UAT environment, and finally promoted to production. Update Sets underpinthispromotion workflow by serving as version-controlled containers that record every customisationmadeduring a development session.

This project integrates both capabilities to address a common data governance challenge inincidentmanagement: the absence of mandatory priority classification when incidents are activelybeingworked on. By implementing a UI Policy to enforce the Priority field and capturing the changewithinan Update Set for subsequent migration, the project demonstrates a complete, professionallystructured approach to ServiceNow platform administration.

## 2.1 The ServiceNow Platform

ServiceNow is a cloud-based IT Service Management (ITSM) platformdesignedtohelporganisations automate, standardise, and optimise their IT workflows. Built on a multi-tenantarchitecture, it provides a unified system of record for managing incidents, service requests, changes, problems, assets, and configurations. Organisations across industries deployServiceNow to reduce operational friction, enforce governance policies, and improvetheconsistency of IT service delivery. The platform's no-code and low-code configurationcapabilities make it accessible to administrators without deep programming backgrounds, whileits scripting engine accommodates complex enterprise customisations.

## 2.2 Incident Management in ServiceNow

The Incident Management module is one of the most widely used applications withinServiceNow. It provides a structured workflow for logging, categorising, assigning, andresolving IT issues reported by end users or detected by monitoring systems. Incidents movethrough a defined lifecycle — from New to In Progress, On Hold, Resolved, and Closed—with each state transition representing a meaningful step in the resolution process. Accuratedata entry at each transition point is critical for SLA compliance, reporting accuracy, workloaddistribution, and effective escalation management.

## 2.3 Business Rule Enforcement and UI Policies

ServiceNow provides multiple mechanisms for enforcing business rules. Server-sidemechanisms such as Business Rules and Data Policies evaluate conditions and constraintsafterform submission, which can result in confusing error messages post-save. UI Policies, bycontrast, operate entirely on the client side, responding dynamically to field changes as theuserinteracts with the form. They can make fields mandatory, read-only, or visible/hiddenbasedonconfigurable conditions, providing immediate, real-time guidance to the user before submissionoccurs. This makes UI Policies the most appropriate tool for enforcement requirementsthatbenefit from immediate user feedback.

### 2.4 Update Sets and Configuration Migration

In a standard ServiceNow deployment, the platform is instantiated across multipleenvironments: a development instance where configurations are built and tested, a UATorstaging instance for validation, and a production instance for live operations. Update Setsarethe native mechanism for transporting configuration changes between these environments. When an Update Set is designated as active ("current"), ServiceNow records every subsequent configuration change — including UI Policies, Business Rules, and formlayouts —intothatcontainer. The completed Update Set can then be exported as XML, transferred, andimportedinto any target instance. This approach ensures consistency, provides an audit trail, andeliminates the risk of manual re-configuration errors in target environments.

## 3. Problem Statement

In organisations relying on ServiceNow for IT incident management, a recurringdataquality issue arises when support agents transition an incident to the "In Progress" state withoutcompleting all fields that are contextually critical at that stage. The Priority field is onesuchfield: it defines the urgency and business impact of the incident, drives SLAtimer activation, determines queue routing in automated workflows, and informs management dashboardsandescalation logic. When Priority is absent at the point of active case management, thedownstream consequences include incorrect SLA calculations, inaccurate reportingmetrics, and potential breaches of service commitments.

Compounding this issue is the challenge of migrating any corrective configurationtoproduction. Without a disciplined Update Set workflow, administrators riskapplyingconfigurations manually across environments, leading to discrepancies between developmentand production instances, inadequate change records, and difficulty rolling back if anissuearises. This project addresses both the enforcement gap and

the migration challenge throughastructured, documented implementation.

## 4. Objectives

The following objectives define the scope and success criteria of this project:

• Design and implement a UI Policy on the ServiceNow Incident table that enforces thePriority field as mandatory when the incident State field is set to "In Progress". • Configure a corresponding UI Policy Action that applies the mandatory attributespecifically to the Priority field under the defined condition.

• Create a dedicated Update Set prior to implementation to ensure that all configurationchanges are systematically captured within a single, named container. • Verify that the Update Set correctly records the UI Policy and UI Policy Actionas customer update entries.

• Export the completed Update Set from the source instance as an XML file for transport. • Import the Update Set into a separate target ServiceNow instance using the RetrievedUpdate Sets module.

## 5. Scope of the Project

The scope of this project is confined to the ServiceNow Incident Management module, specifically the enforcement of a single mandatory field condition using a UI Policyandthemigration of that configuration using an Update Set. The implementation targetsanyServiceNow instance running on the Rome release or later, including Personal DeveloperInstances (PDIs) provisioned through the ServiceNow Developer Program.

The solution applies to all users who interact with the Incident formand have the abilitytomodify the State field. It does not require changes to any other module, table, or application, and it does not rely on custom scripts, business rules, or workflow modifications. The UpdateSet migration scope covers the source and one target instance, demonstrating the standardsingle-hop promotion pattern used in most enterprise deployments.

## 6. Tools and Technologies Used

| | |
|---|---|
| ServiceNow Platform (PDI) | The cloud-based ITSM platform on which all development, configuration, and testing activities are performed. APersonalDeveloper |

| | |
|---|---|
| | Instance is used to provide a safe, isolated developmentenvironment. |
| Incident Table (in | The application table on which the UI Policy is created. It representssupport cases and contains the State and Priority fields central tothisproject. |
| UI Policies | Client-side configuration objects that dynamically adjust formfieldbehaviour — such as mandatory status, visibility, and read-onlybased on conditions defined by the administrator. |
| UI Policy Ac | Child records of a UI Policy that specify the exact field-level constraintto enforce. Each action targets one field and defines its mandatory,visible, or read-only setting. |
| Update | Sets Version-controlled configuration containers that recordallcustomisati made while the set is designated as current. Usedtotransport configuratior between ServiceNow instances. |
| XML Export / Impo | serialisation format used to export an Update Set fromthe instance and import it into the target instance for previewanc |
| Web Browser | web browser (Google Chrome recommended) |

IT

state—

source

access the

## 7. Project Phases

| Phase 1 |
|---|

The Identification Phase establishes the problem context and justifies the needforatechnical solution. The specific issue — incidents progressing to the "In Progress" state withouta Priority value — is identified through analysis of common incident data quality failuresinServiceNow environments. The

suitability of a UI Policy as a client-side enforcementmechanism is confirmed, and the need for a structured Update Set migration workflowisarticulated. The phase concludes with a formally defined project scope and a clear statement ofthe business value the solution will deliver.

## Phase 2

Requirement Analysis translates the identified problem into specific, testablerequirements. The primary functional requirement is defined: when an incident's Statefieldisset to "In Progress", the Priority field must be marked as mandatory and must prevent formsubmission if empty. Technical requirements include access to a ServiceNowinstancewithadministrative privileges, the ability to create UI Policies on the Incident table, andaccesstothe Update Sets module. The requirement for the solution to operate without customscriptingisalso documented as a non-functional requirement, ensuring long-termmaintainabilityandupgrade compatibility.

## Phase 3

The Planning Phase organises the implementation into a seven-step sequencewithdefined inputs, outputs, and acceptance criteria for each step. Key milestones are identified:Update Set creation and activation, successful UI Policy configuration, Update Set captureverification, XML export, target instance import, and post-commit validation. Risksareassessed — including PDI hibernation, Dot Walk resolution failures, and import conflicts—and mitigation strategies are documented. The planning phase ensures that the implementationproceeds systematically and that each step can be independently verified before proceedingtothe next.

## Phase 4

The Design Phase defines the logical structure of the solution. The UI Policyisdesigned with a single, precise condition: State equals "In Progress" (integer value 2intheServiceNow incident.state field). The UI Policy Action is designed to target the priorityfieldexclusively, with the Mandatory attribute set to True, and both Visible and Read Onlyset to"Leave alone" to avoid unintended side effects. The Update Set is designed as a project-scopedcontainer with a descriptive naming convention. The expected migration workflowisdocumented as a linear sequence: capture, complete, export, import, preview, commit, andvalidate.

Phase 5

The Development Phase encompasses the hands-on implementationof allconfigured components within the ServiceNow instance. All seven implementationstepsareexecuted in sequence, with screenshots captured at each key stage as evidence of correctconfiguration. The phase includes the creation of the Update Set, the configurationof theUIPolicy and its associated Action, and the full export-import migration cycle. Thephaseconcludes when the UI Policy is confirmed as active in the target instance and the TransformLog — in this context, the Update Set commit log — reports a successful outcome.

Phase 6

The Documentation Phase produces this complete academic report, consolidating all phases, implementation steps, testing procedures, results, and supporting evidenceintoasingle structured document. The report is written in formal academic language, usesaconsistent heading hierarchy, maintains justified paragraph alignment, and includes screenshotplaceholders at all relevant implementation points. The documentation is structuredforsubmission in an academic or internship context and is sufficiently detailed to serveasareproducible reference guide for any administrator wishing to replicate the configuration.

Phase 7

The Demonstration Phase presents the working solution to an examiner, supervisor, or review panel. The demonstration covers four scenarios: opening an incident andchangingthe State to "In Progress" to trigger the mandatory Priority indicator; attempting to save withouta Priority value to confirm that the validation prevents submission; completing the Priorityfieldand saving successfully; and presenting the Update Set XML file alongside the post-commit UIPolicy list in the target instance. All four scenarios are executed live or captured inavideowalkthrough, confirming that the solution meets every stated objective.

# 8. System Architecture and Workflow

## 8.1 UI Policy Execution Workflow

When a user opens an Incident record in ServiceNow, the platformloads all UI Policiesassociated with

the Incident table into the browser. Each policy's condition is evaluatedclient-side as the user modifies form fields. In this project, when the user changes the State fieldto"In Progress", the UI Policy condition becomes true and the browser immediately appliestheUI Policy Action: the Priority field is marked as mandatory, displaying a red asteriskandtriggering a validation error if the user attempts to save without completing the field. If theState is subsequently changed to any other value, the condition becomes false andthemandatory indicator is removed, demonstrating the dynamic, reversible nature of UI Policyenforcement.

## 8.2 Update Set Capture and Migration Workflow

The Update Set workflow follows a linear, multi-stage process. First, an administratorcreates a new Update Set and designates it as current. All subsequent configuration changes—including the creation of the UI Policy record and its UI Policy Action —are automaticallyrecorded as customer update entries within the active Update Set. Once all changesarecomplete, the Update Set's state is changed to Complete, which prevents further captureandprepares it for export. The administrator uses the Export to XML function to downloadtheUpdate Set as a self-contained XML file.

**MIGRATION WORKFLOW OVERVIEW**

Create Update Set → Make Current → Configure UI Policy →Add UI Policy Action↓

Verify Capture → Set Status to Complete → Export to XML↓

Import XML to Target Instance → Preview → Commit →Validat

# 9. Implementation Steps

| STEP | |
|---|---|
| | **Create a Dedicated Update Set**<br><br>Navigate to System Update Sets → Local Update Sets and click New. In the Name enter a descriptive identifier such as "UI Policy – Enforce Priority on InIncidents". Set the Description field to document the purpose of this Update Set for futurereference. Leave the as "In Progress" and click Submit to save the record. |

field,

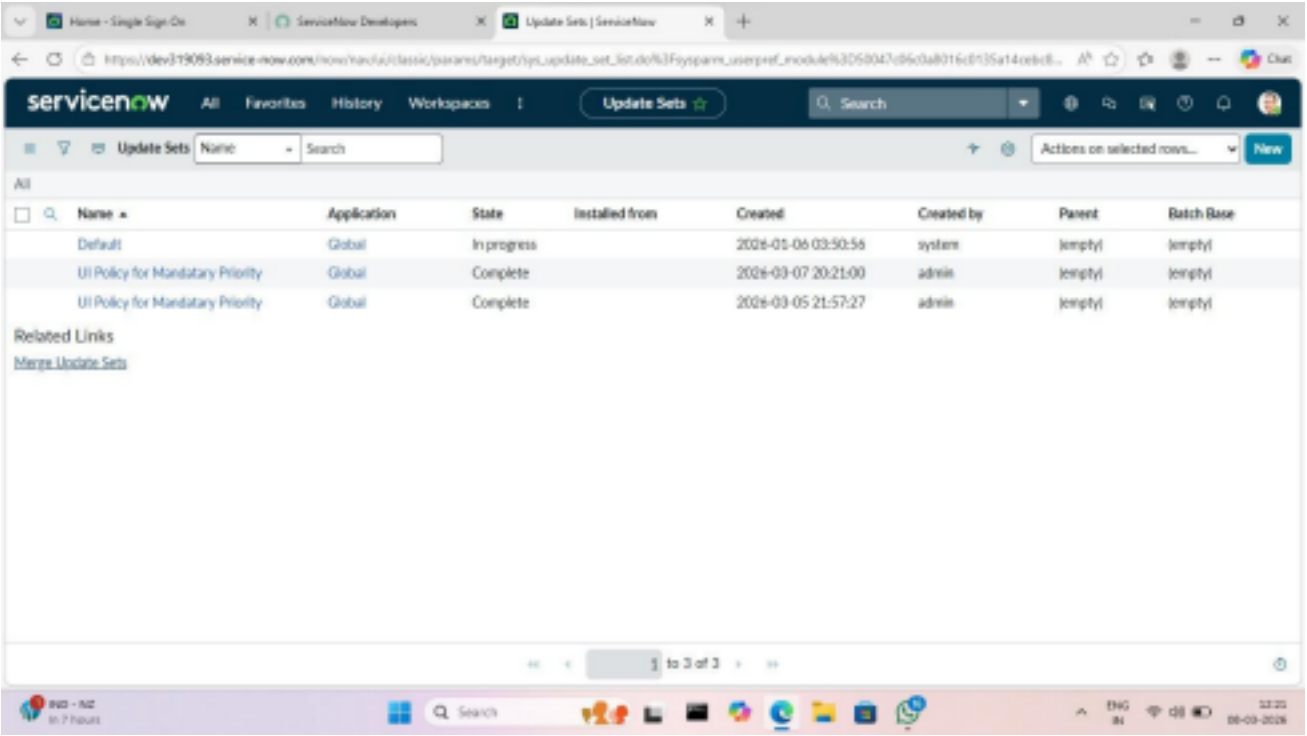


*Figure 1 – New Update Set record showing Name, Description, and State fields*

| STEP | **Designate the Update Set as Current** |
|---|---|
| | Open the Update Set record that was just created. In the form header, click the MakeCurrent Observe that the ServiceNow banner at the top of the interfacenow<br><br>displays the Update Set name as the active set. This confirms that all subsequentconfiguratic changes will be captured within this Update Set. Verify the active set namebefore proceedin |

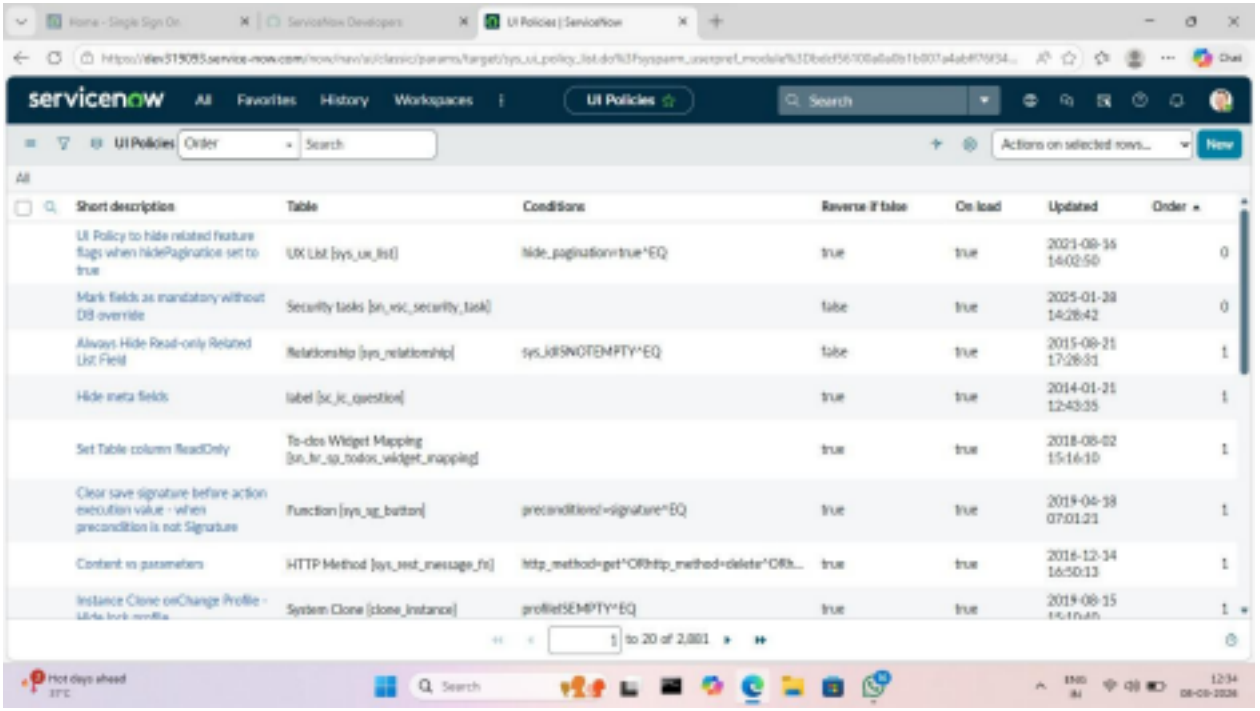


*Figure 2 – ServiceNow instance header confirming the active Update Set name*

| STEP | **Create a New UI Policy**<br><br>Navigate to System UI → UI Policies and click New. In the Table field, select Incident[incident] from the list. Enter a clear Name such as "Enforce Priority – In Progress<br><br>Ensure that the Active checkbox is checked. Also check the On load checkbox to ensurethep evaluates when the Incident form is first opened by a user, not only whenchanged during the |
|---|---|

State".

a fieldis

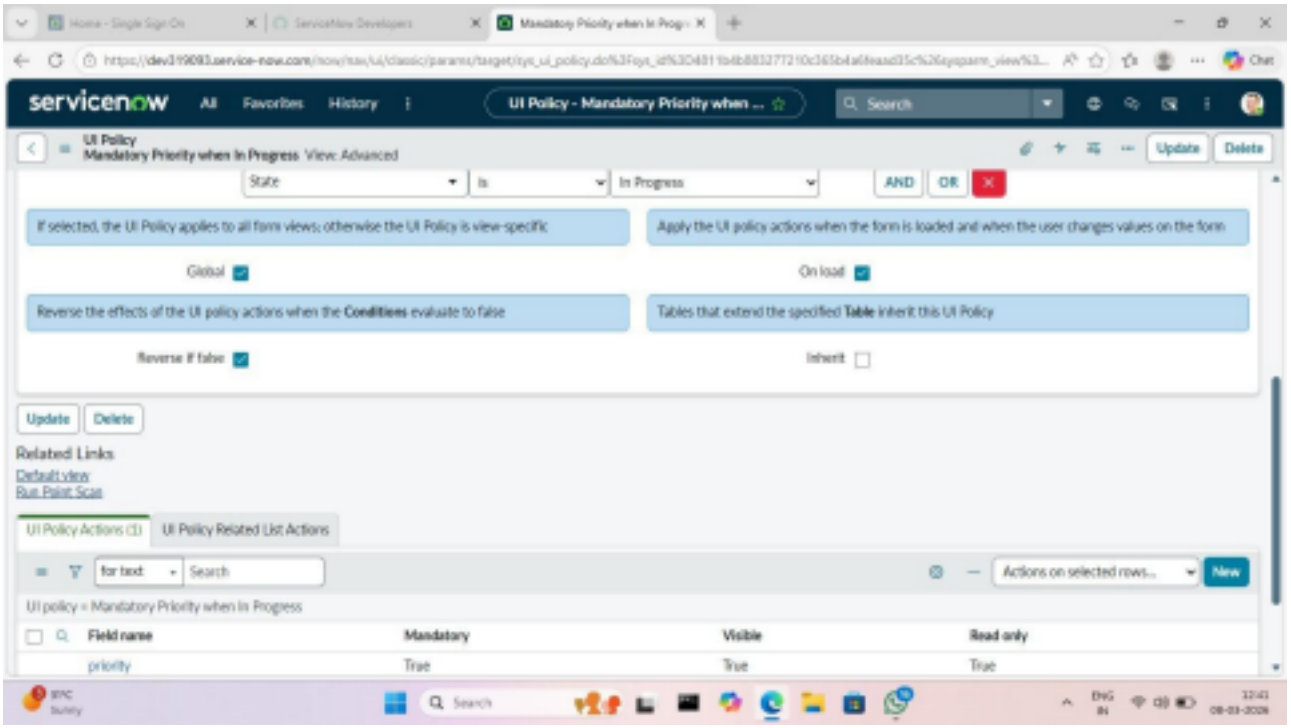


*Figure 3 – UI Policy form showing Table set to Incident, Name, Active, and On load fields*

| STEP | **Configure the UI Policy Condition**<br><br>In the Conditions section of the UI Policy form, click the filter builder to add a condition. Se Field to State, the Operator to is, and the Value to In Progress. The conditionstring<br><br>should read: State is In Progress. This ensures the policy activates exclusivelyState field val equals In Progress and remains inactive for all other state values. ClickSubmit to save the condition. |
|---|---|

whenthe

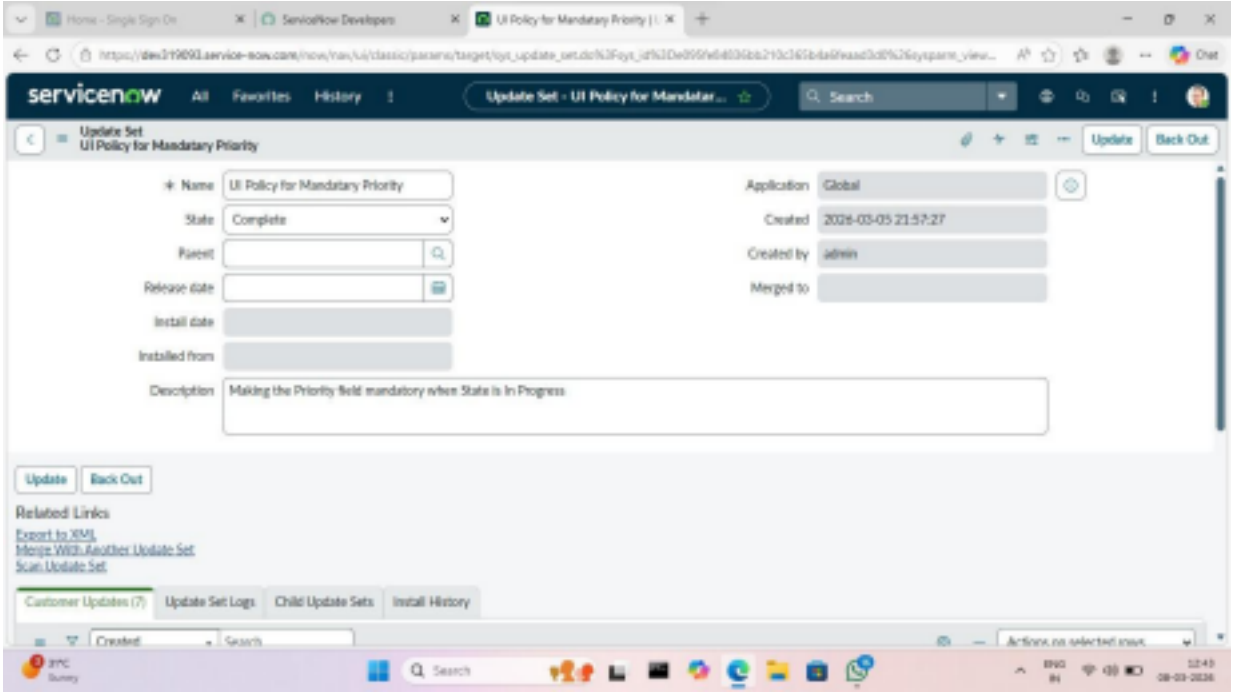


*Figure 4 – UI Policy Conditions section showing: State | is | In Progress*

| | |
|---|---|
| STEP | **Add a UI Policy Action for the Priority Field**<br><br>Scroll to the UI Policy Actions related list at the bottom of the UI Policy formandclickNew.<br>the Field field, select Priority. Set the Mandatory dropdown to True. Set boththe<br><br>Visible and Read Only dropdowns to Leave alone to ensure only the mandatory attributeisap<br>without affecting field visibility or editability. Click Submit to save the UI PolicyAction rec |

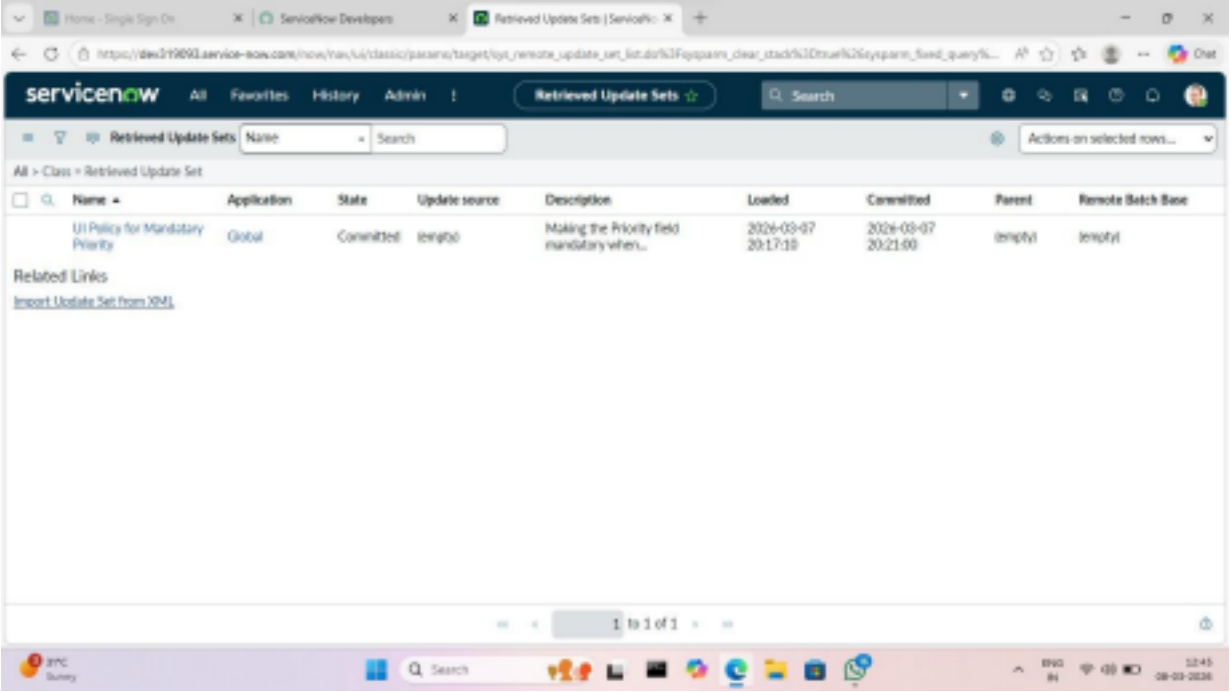


*Figure 5 – UI Policy Action form: Field = Priority, Mandatory = True, Visible = Leave alone, Read Only =Leavealone*

| STEP | **Save and Activate the UI Policy**<br><br>Return to the parent UI Policy record and confirm that the UI Policy Action is visible related list. Verify that the Active checkbox on the UI Policy record is checked. If theformis mode, click Update to save all changes. The UI Policy is nowactive andwillenforce the Prio mandatory rule on the Incident form whenever the State is In Progress. |
|---|---|

inthe

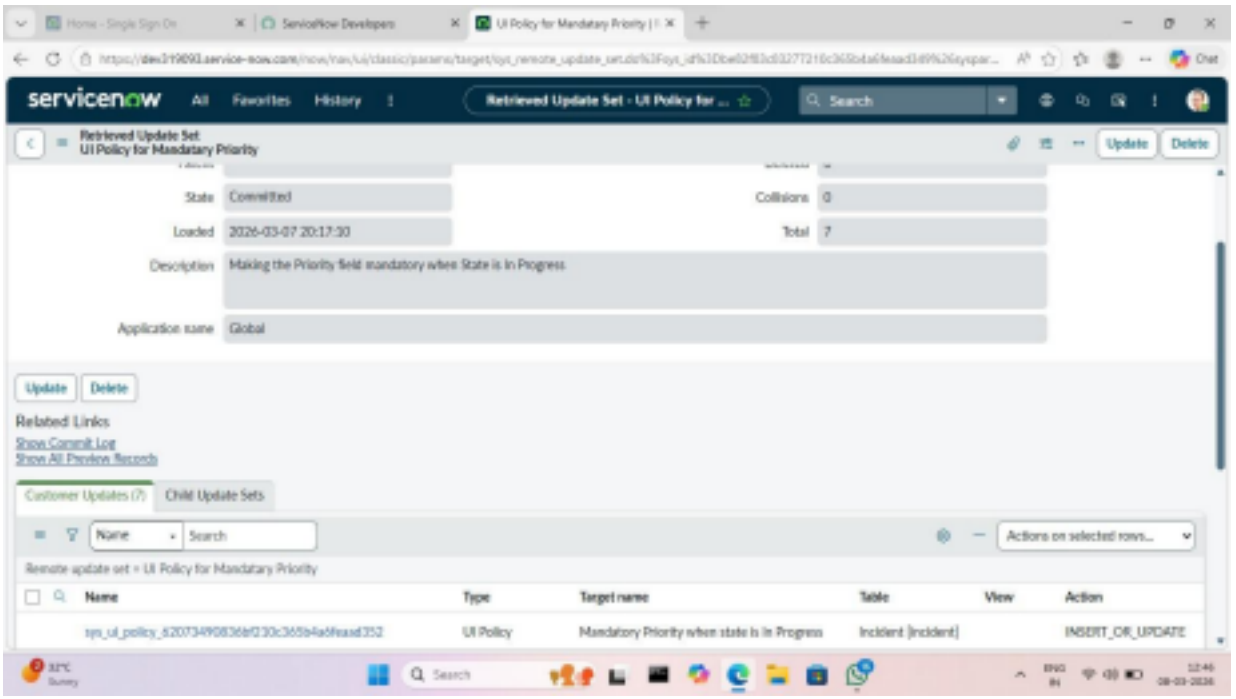


*Figure 6 – UI Policy record in active state with UI Policy Action visible in related list*

| STEP | **Verify Update Set Capture**<br><br>Navigate to System Update Sets → Local Update Sets and open the active UpdateSet.Scroll the Customer Updates related list at the bottom of the record. Confirmthat<br><br>entries are present: one corresponding to the UI Policy record and one correspondingtotheU Policy Action record. Both entries confirm that the configuration changes havesuccessfully captured and are ready for export. |
|---|---|

two

been

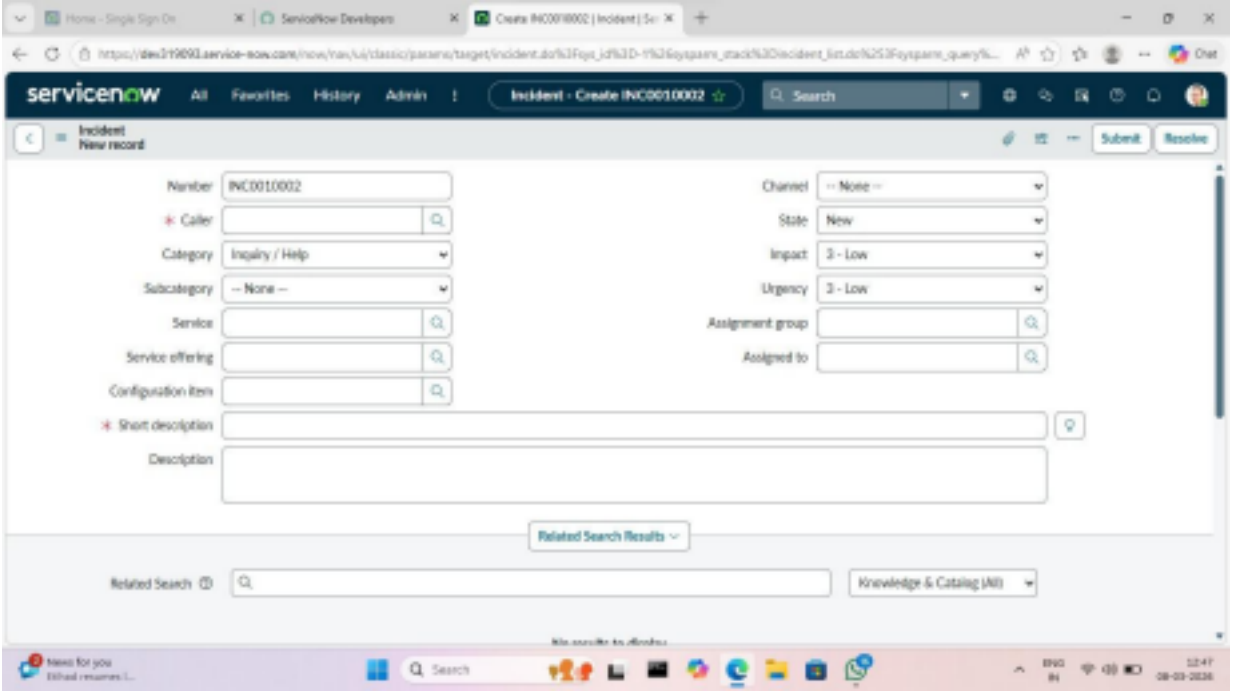


*Figure 7 – Customer Updates related list showing UI Policy and UI Policy Action entries*

## 10. Advantages

| | |
|---|---|
| Improved Data Acc | Mandat eliminatesa common sou of operational data and th |
| Real-Time User Feedback | Because UI P Themandatory i beforethe user |
| No CustomRequired | The entire en Scrip no-code configuration. Th of scripting errors, and en |
| Better WorkflowEnforcement | By tying field requiremen actively guides agents thr supervisory review of inc |
| Easy ConfigurationMigration | The Update Set framewo pathway. The same XML ensuring consistent enfor environments. |
| Auditability | Every configuration customer update log reco modified, supporting gov |

| | |
|---|---|
| Low-Code Implementation | The s<br>implement ormaintain. A<br>platformknowledge can r<br>administrative interface. |

target

and

and

usingthe

the

## 11. Limitations

- UI Policies are client-side only and execute within the browser. They can be bypassedby API calls, import operations, or any process that creates or updates records withoutloading the ServiceNow form. For server-side enforcement that cannot be bypassed, aData Policy or Business Rule must be used in addition to the UI Policy.
- UI Policies apply only to the form view in the standard ServiceNowinterface. Theydonot affect list editing, mobile interfaces (without additional configuration), orintegrations that write data directly to the database layer.
- Update Sets capture configuration changes but do not capture data. Changes torecordsin operational tables — such as incident data, user records, or knowledge articles —arenot included in an Update Set and must be migrated by other means if required.
- Personal Developer Instances used for development may enter a hibernation stateafteraperiod of inactivity. If the instance hibernates during an active development session, theUpdate Set may need to be reviewed to confirm that all changes were capturedbeforethe instance went offline.

## 12. Future Enhancements

- Implement a complementary Data Policy on the Incident table to enforce the Prioritymandatory rule at the server side, ensuring that the constraint is applied even for API-based or import-based record updates that bypass the form entirely.
- Extend the UI Policy framework to enforce additional contextually relevant fields at other state transitions. For example, making the Resolution Notes and ResolutionCodefields mandatory when State changes to Resolved, and requiring Category andSubcategory before an incident can be Assigned.
- Configure an automated notification or task assignment trigger using a FlowDesignerflow that fires when an incident transitions to "In Progress" without a Priority value, asa compensating control for the API bypass scenario described in Section 16.
- Explore the use of Automated Test Framework (ATF) test suites to automate theregression testing of UI Policies across multiple Incident scenarios, reducing the timerequired for manual validation after each Update Set migration.

## 13. Conclusion

This project has successfully demonstrated two core ServiceNowadministrationcompetencies: the dynamic enforcement of mandatory fields using UI Policies, andthestructured migration of configuration changes using Update Sets. Both capabilities wereapplied to a realistic and practically significant use case—ensuring that everyincidentreaching the "In Progress" state carries a Priority value, which is essential for SLAcompliance, workload management, and accurate reporting.

The UI Policy implementation confirmed that meaningful business rule enforcement canbeachieved through entirely declarative, no-code configuration. The policy operates inreal time, responds dynamically to user interactions, and deactivates automatically when its conditionisno longer satisfied. This approach is maintainable, upgrade-safe, and accessibletoadministrators without scripting expertise.

The Update Set workflow demonstrated that ServiceNow's native configurationtransportmechanism is reliable, auditable, and straightforward to operate. The complete export-import-preview-commit cycle produced identical behaviour in the target instance without manual re-configuration, validating the

Update Set as the correct tool for enterprise-scale configurationmanagement.

Collectively, the project illustrates a professional, methodical approach to ServiceNowplatform administration: identifying a data quality problem, designing a precise, minimalsolution, implementing it within a controlled development environment, and migratingit toatarget instance through a documented, reversible process. This methodology is directlyapplicable to real-world enterprise ServiceNow deployments and represents a foundationalcompetency for any practitioner working in the ServiceNow ecosystem.

*— End of Project Documentation —*